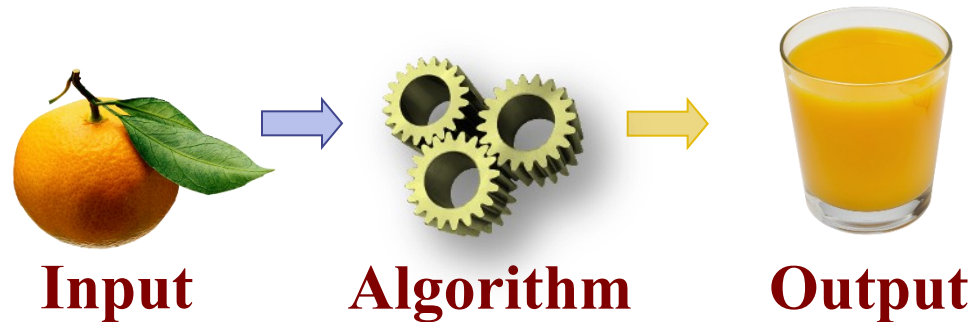


Analysis of Algorithms



Running Time

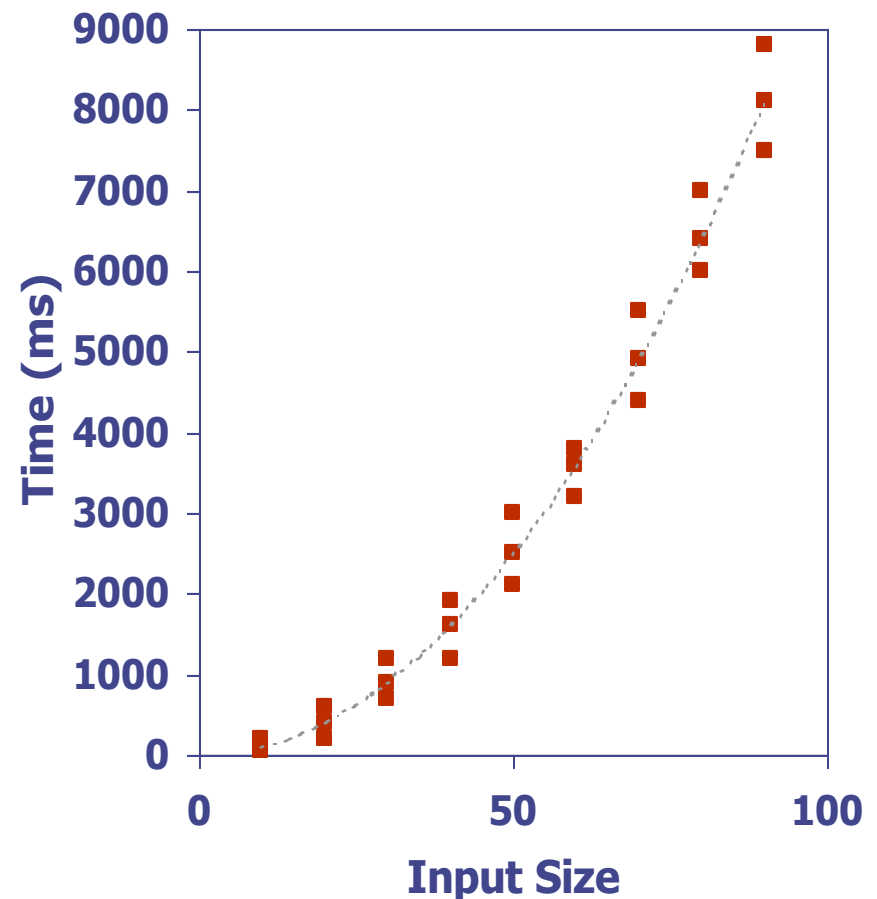
- ❑ Most algorithms transform input objects into output objects.
- ❑ The running time of an algorithm typically grows with the input size.
- ❑ Average case time is often difficult to determine.

Experimental Studies

- Write a program implementing the algorithm
- Run the program with inputs of varying size and composition, noting the time needed:

```
from time import time
start_time = time( )
run algorithm
end_time = time( )
elapsed = end_time - start_time
```

- Plot the results



Limitations of Experiments

- ❑ It is necessary to implement the algorithm, which may be difficult
- ❑ Results may not be indicative of the running time on other inputs not included in the experiment.
- ❑ In order to compare two algorithms, the same hardware and software environments must be used



Theoretical Analysis



- ❑ Uses a high-level description of the algorithm instead of an implementation
- ❑ Characterizes running time as a function of the input size, n .
- ❑ Takes into account all possible inputs
- ❑ Allows us to evaluate the speed of an algorithm independent of the hardware/software environment

Pseudocode

- ❑ High-level description of an algorithm
- ❑ More structured than English prose
- ❑ Less detailed than a program
- ❑ Preferred notation for describing algorithms
- ❑ Hides program design issues

Pseudocode Details



- Control flow
 - **if ... then ... [else ...]**
 - **while ... do ...**
 - **repeat ... until ...**
 - **for ... do ...**
 - Indentation replaces braces

- Method declaration

Algorithm *method* (*arg* [, *arg...*])

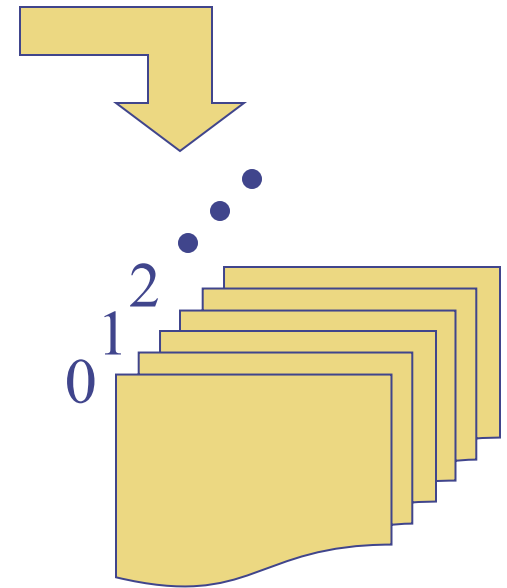
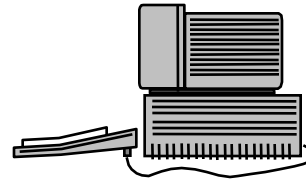
Input ...

Output ...

- Method call
method (*arg* [, *arg...*])
- Return value
return *expression*
- Expressions:
 - ← Assignment
 - = Equality testing
 - n^2 Superscripts and other mathematical formatting allowed

The Random Access Machine (RAM) Model

- A **CPU**

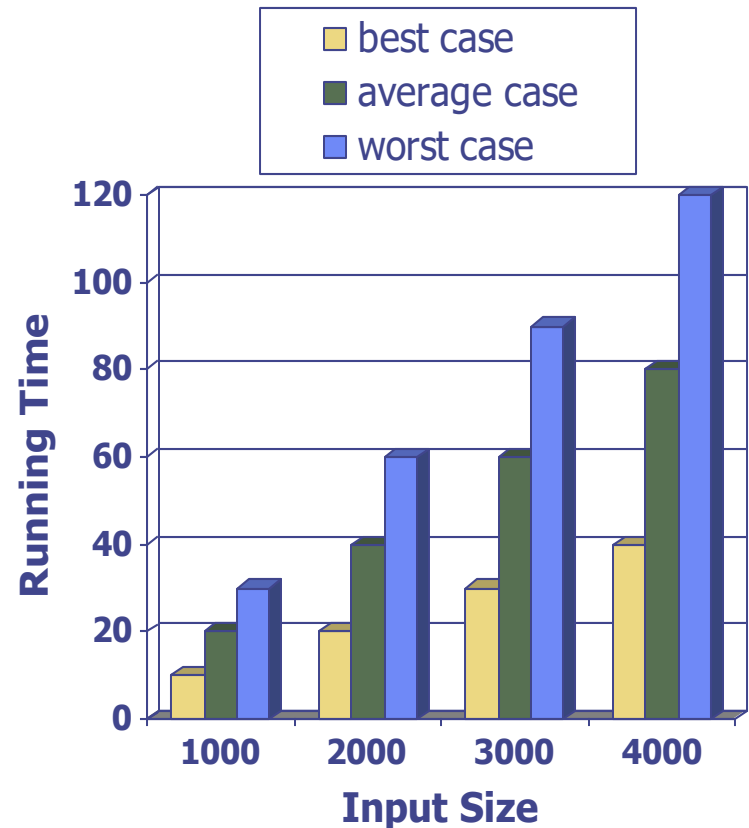


- An potentially unbounded bank of **memory** cells, each of which can hold an arbitrary number or character

- ◆ Memory cells are numbered and accessing any cell in memory takes unit time.

Worst Case Execution Time

- Average case time is often difficult to determine.
- We focus on the worst case running time.
 - Easier to analyze
 - Crucial to applications such as games, finance and robotics

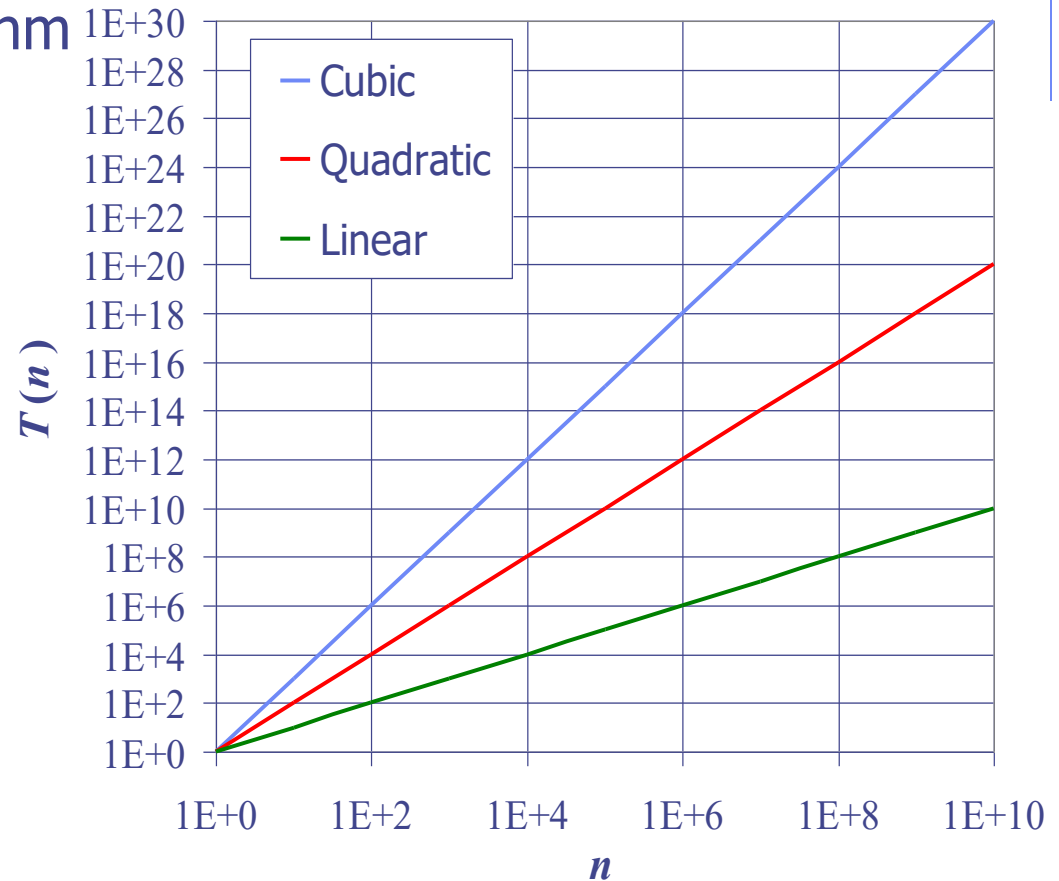


Seven Important Functions

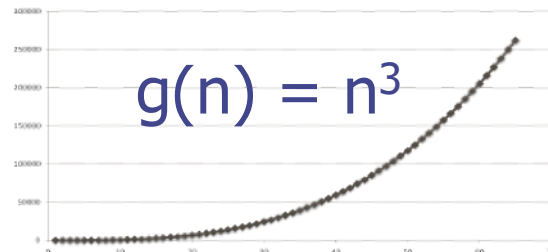
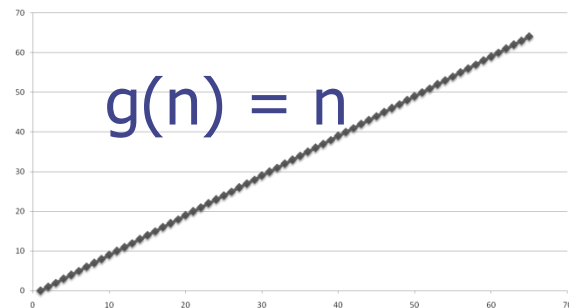
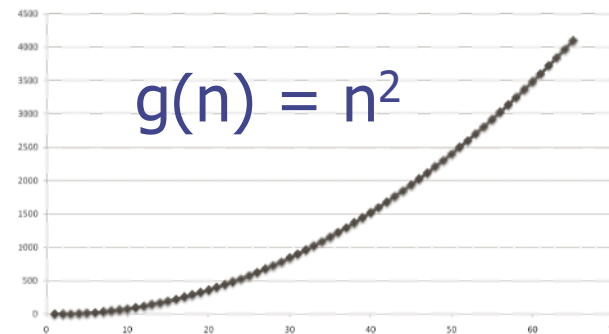
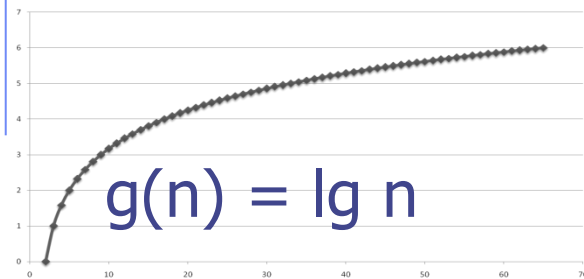
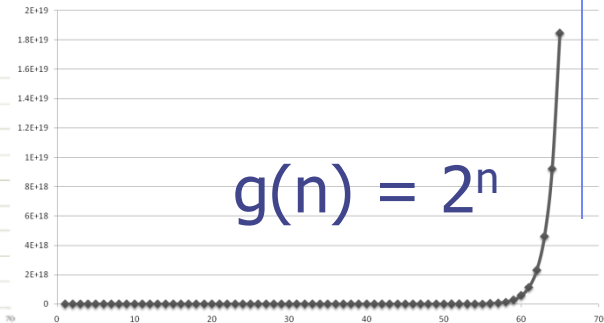
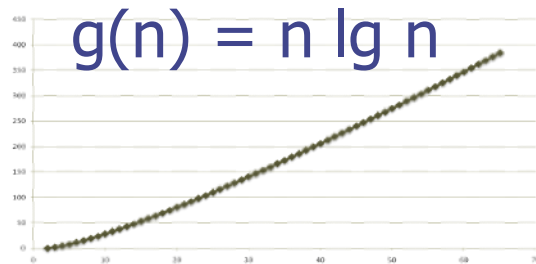
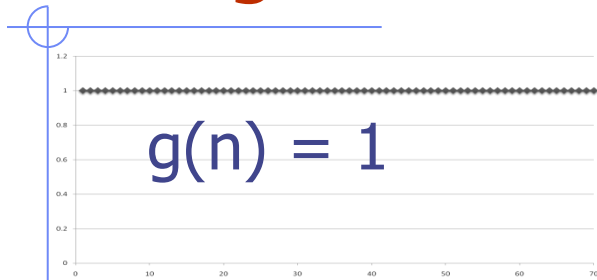
- Seven functions that often appear in algorithm analysis:

- Constant ≈ 1
- Logarithmic $\approx \log n$
- Linear $\approx n$
- N-Log-N $\approx n \log n$
- Quadratic $\approx n^2$
- Cubic $\approx n^3$
- Exponential $\approx 2^n$

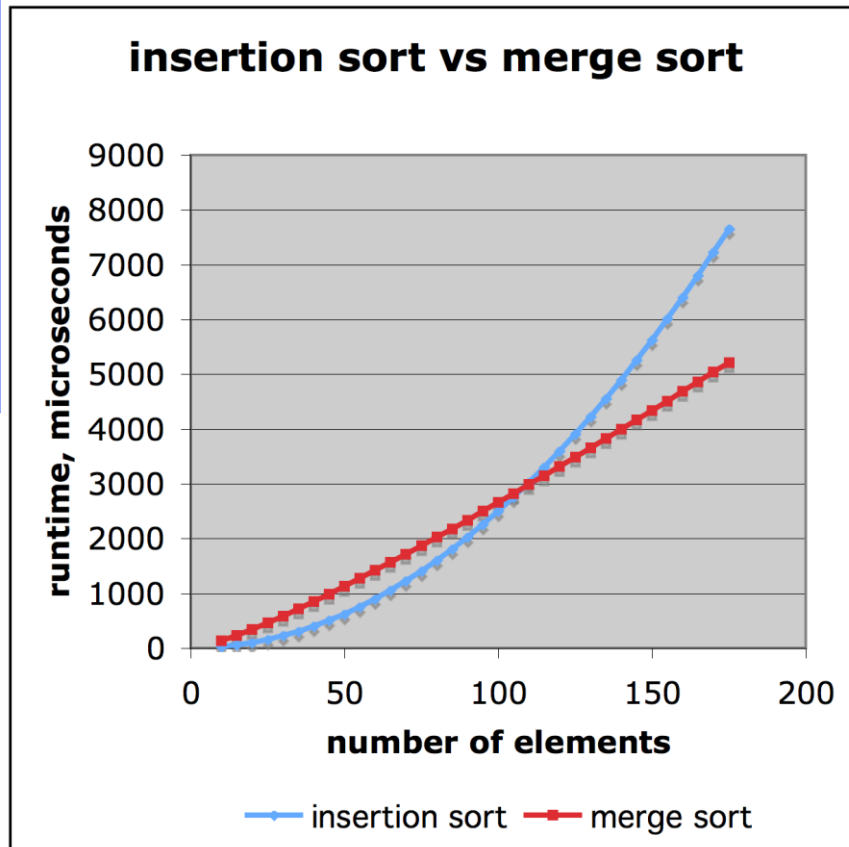
- In a log-log chart, the slope of the line corresponds to the growth rate



Functions Graphed Using “Normal” Scale



Comparison of Two Algorithms



insertion sort is
 $n^2 / 4$

merge sort is
 $2 n \lg n$

sort a million items?

insertion sort takes
roughly **70 hours**

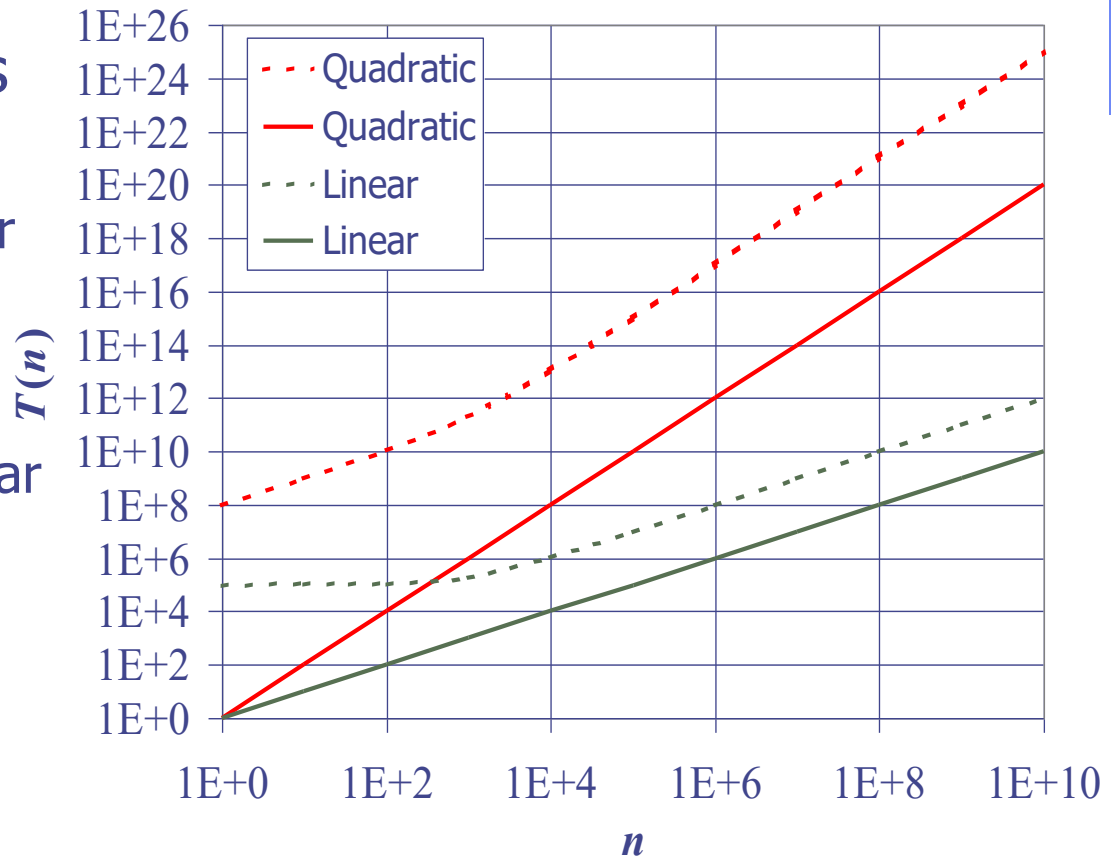
while

merge sort takes
roughly **40 seconds**

This is a slow machine, but if
100 x as fast then it's **40 minutes**
versus less than **0.5 seconds**

Constant Factors

- The growth rate is not affected by
 - constant factors or
 - lower-order terms
- Examples
 - $10^2n + 10^5$ is a linear function
 - $10^5n^2 + 10^8n$ is a quadratic function



Primitive Operations



- Basic computations performed by an algorithm
 - Identifiable in pseudocode
 - Largely independent from the programming language
 - Exact definition not important (we will see why later)
 - Assumed to take a constant amount of time in the RAM model
- Examples:
 - Evaluating an expression
 - Assigning a value to a variable
 - Indexing into an array
 - Calling a method
 - Returning from a method

Counting Primitive Operations

- By inspecting the pseudocode, we can determine the maximum number of primitive operations executed by an algorithm, as a function of the input size

```
1  def find_max(data):
2      """Return the maximum element from a nonempty Python list."""
3      biggest = data[0]           # The initial value to beat
4      for val in data:           # For each value:
5          if val > biggest        # if it is greater than the best so far,
6              biggest = val       # we have found a new best (so far)
7      return biggest             # When loop ends, biggest is the max
```

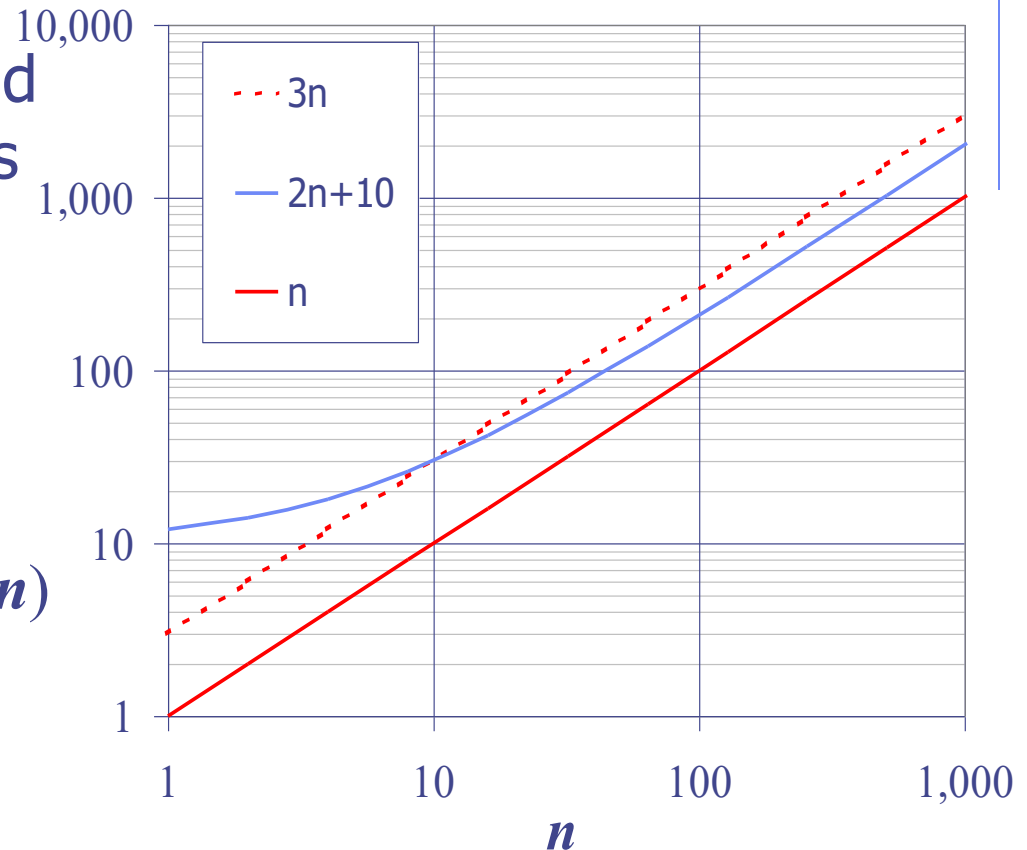
- linear

Big-Oh Notation

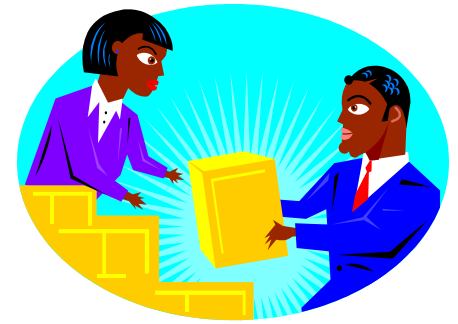
- Given functions $f(n)$ and $g(n)$, we say that $f(n)$ is $O(g(n))$ if there are positive constants c and n_0 such that

$$f(n) \leq cg(n) \text{ for } n \geq n_0$$

- Example: $2n + 10$ is $O(n)$
 - $2n + 10 \leq cn$
 - $(c - 2)n \geq 10$
 - $n \geq 10/(c - 2)$
 - Pick $c = 3$ and $n_0 = 10$



Big-Oh Examples



◆ $7n-2$

$7n-2$ is $O(n)$

need $c > 0$ and $n_0 \geq 1$ such that $7n-2 \leq c \cdot n$ for $n \geq n_0$

this is true for $c = 7$ and $n_0 = 1$

■ $3n^3 + 20n^2 + 5$

$3n^3 + 20n^2 + 5$ is $O(n^3)$

need $c > 0$ and $n_0 \geq 1$ such that $3n^3 + 20n^2 + 5 \leq c \cdot n^3$ for $n \geq n_0$

this is true for $c = 4$ and $n_0 = 21$

■ $3 \log n + 5$

$3 \log n + 5$ is $O(\log n)$

need $c > 0$ and $n_0 \geq 1$ such that $3 \log n + 5 \leq c \cdot \log n$ for $n \geq n_0$

this is true for $c = 8$ and $n_0 = 2$

Asymptotic Algorithm Analysis

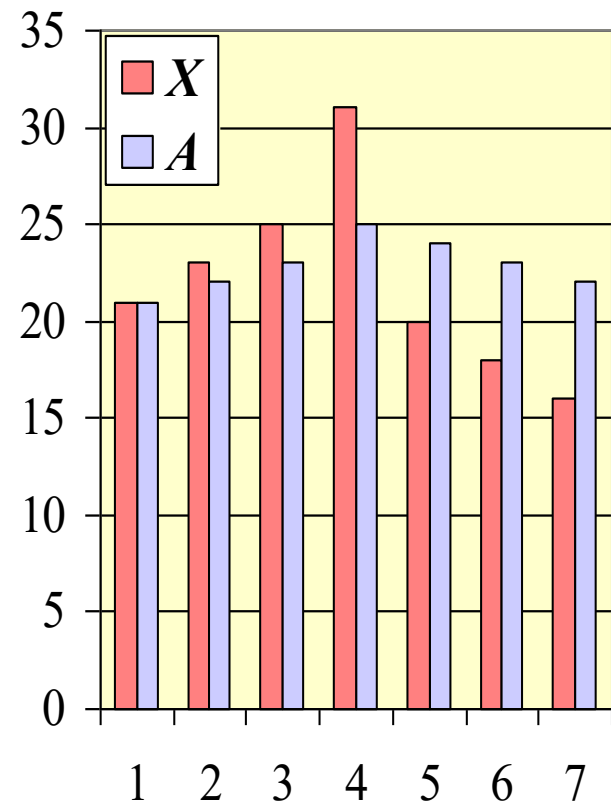
- The asymptotic analysis of an algorithm determines the running time in big-Oh notation
- To perform the asymptotic analysis
 - We find the worst-case number of primitive operations executed as a function of the input size
 - We express this function with big-Oh notation
- Example:
 - We say that algorithm `find_max` “runs in $O(n)$ time”
- Since constant factors and lower-order terms are eventually dropped anyhow, we can disregard them when counting primitive operations

Computing Prefix Averages

- We further illustrate asymptotic analysis with two algorithms for prefix averages
- The i -th prefix average of an array X is average of the first $(i + 1)$ elements of X :

$$A[i] = (X[0] + X[1] + \dots + X[i]) / (i + 1)$$

- Computing the array A of prefix averages of another array X has applications to financial analysis



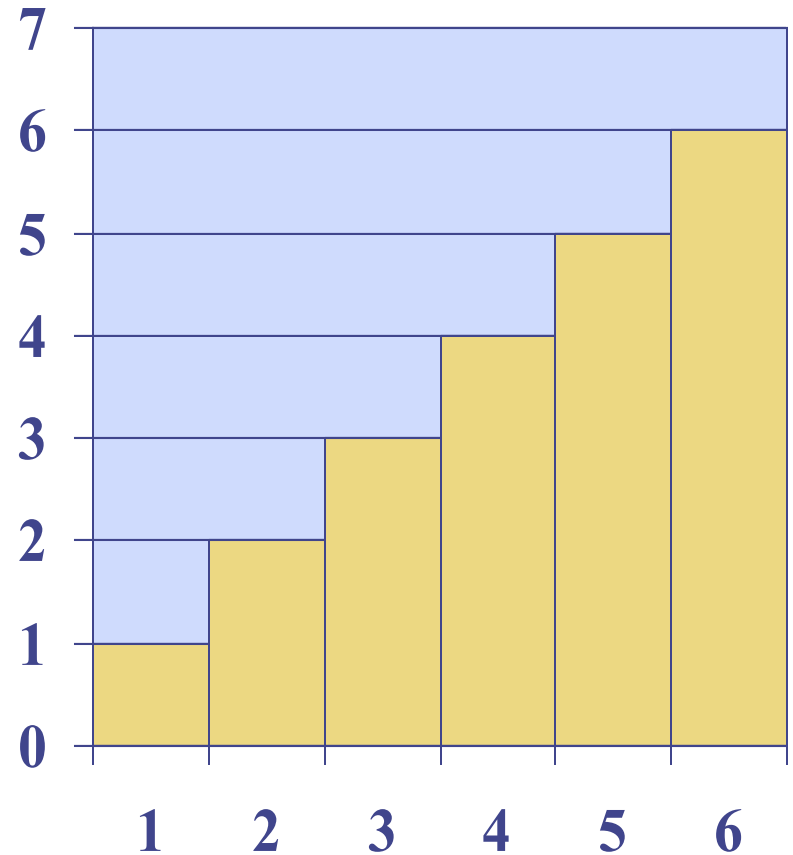
Prefix Averages (Quadratic)

- ◆ The following algorithm computes prefix averages in quadratic time by applying the definition

```
1  def prefix_average1(S):
2      """Return list such that, for all j, A[j] equals average of S[0], ..., S[j]."""
3      n = len(S)
4      A = [0] * n                # create new list of n zeros
5      for j in range(n):
6          total = 0              # begin computing S[0] + ... + S[j]
7          for i in range(j + 1):
8              total += S[i]
9          A[j] = total / (j+1)    # record the average
10     return A
```

Arithmetic Progression

- The running time of *prefixAverage1* is $O(1 + 2 + \dots + n)$
- The sum of the first n integers is $n(n + 1) / 2$
 - There is a simple visual proof of this fact
- Thus, algorithm *prefixAverage1* runs in $O(n^2)$ time



Prefix Averages 2 (Looks Better)

- ◆ The following algorithm uses an internal Python function to simplify the code

```
1 def prefix_average2(S):
2     """ Return list such that, for all j, A[j] equals average of S[0], ..., S[j]. """
3     n = len(S)
4     A = [0] * n                # create new list of n zeros
5     for j in range(n):
6         A[j] = sum(S[0:j+1]) / (j+1)  # record the average
7     return A
```

- ◆ Algorithm ***prefixAverage2*** still runs in $O(n^2)$ time!

Prefix Averages 3 (Linear Time)

- ◆ The following algorithm computes prefix averages in linear time by keeping a running sum

```
1 def prefix_average3(S):
2     """Return list such that, for all j, A[j] equals average of S[0], ..., S[j]."""
3     n = len(S)
4     A = [0] * n                # create new list of n zeros
5     total = 0                  # compute prefix sum as S[0] + S[1] + ...
6     for j in range(n):
7         total += S[j]          # update prefix sum to include S[j]
8         A[j] = total / (j+1)   # compute average based on current sum
9     return A
```

- ◆ Algorithm *prefixAverage3* runs in $O(n)$ time

Relatives of Big-Oh



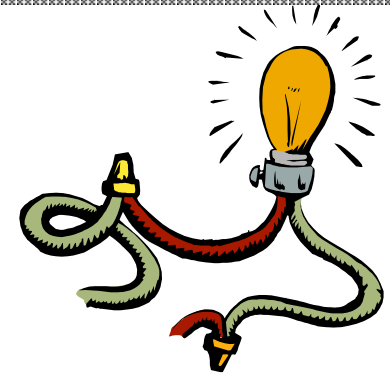
◆ **big-Omega**

- $f(n)$ is $\Omega(g(n))$ if there is a constant $c > 0$ and an integer constant $n_0 \geq 1$ such that $f(n) \geq c \cdot g(n)$ for $n \geq n_0$

◆ **big-Theta**

- $f(n)$ is $\Theta(g(n))$ if there are constants $c' > 0$ and $c'' > 0$ and an integer constant $n_0 \geq 1$ such that $c' \cdot g(n) \leq f(n) \leq c'' \cdot g(n)$ for $n \geq n_0$

Intuition for Asymptotic Notation



Big-Oh

- $f(n)$ is $O(g(n))$ if $f(n)$ is asymptotically **less than or equal** to $g(n)$

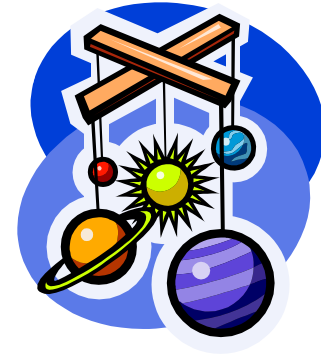
big-Omega

- $f(n)$ is $\Omega(g(n))$ if $f(n)$ is asymptotically **greater than or equal** to $g(n)$

big-Theta

- $f(n)$ is $\Theta(g(n))$ if $f(n)$ is asymptotically **equal** to $g(n)$

Example Uses of the Relatives of Big-Oh



- $5n^2$ is $\Omega(n^2)$

$f(n)$ is $\Omega(g(n))$ if there is a constant $c > 0$ and an integer constant $n_0 \geq 1$ such that $f(n) \geq c \cdot g(n)$ for $n \geq n_0$

let $c = 5$ and $n_0 = 1$

- $5n^2$ is $\Omega(n)$

$f(n)$ is $\Omega(g(n))$ if there is a constant $c > 0$ and an integer constant $n_0 \geq 1$ such that $f(n) \geq c \cdot g(n)$ for $n \geq n_0$

let $c = 1$ and $n_0 = 1$

- $5n^2$ is $\Theta(n^2)$

$f(n)$ is $\Theta(g(n))$ if it is $\Omega(n^2)$ and $O(n^2)$. We have already seen the former, for the latter recall that $f(n)$ is $O(g(n))$ if there is a constant $c > 0$ and an integer constant $n_0 \geq 1$ such that $f(n) \leq c \cdot g(n)$ for $n \geq n_0$

Let $c = 5$ and $n_0 = 1$